武汉理工大学数统学院数学系课程实验报告

课 程 名 称：数字图像处理

班级		**日　期**		**成绩评定**	
姓名		**实验室**		**老师签名**	
实验名称	实验一 图像增强的空域方法实现				
所用软件	Python 3.9.7、NumPy、Pillow、Matplotlib；MATLAB R2023b（仅均衡化对照）				
实验目的及内容	目的：掌握空域图像增强；内容：完成指导书编号 1、3、4 中的直方图、边缘锐化及平滑滤波实验，详见附页。				
实验原理步骤、核心代码	公式、实验步骤与实际运行的核心源代码详见附页。				

实验结果及分析	结果图表、实测指标及分析见附页。 （写不完时，可另附稿纸。）

备注：不交此报告者，本次实验为“不合格”。

实验一 图像增强的空域方法实现（附页）

实验条件、数据来源与复现设置

软件：Python 3.9.7、NumPy 2.0.2、Pillow 11.2.1、Matplotlib 3.9.4、scikit-image 0.24.0（仅用于 PSNR/SSIM）、pytest 8.4.2；另用 MATLAB R2023b Image Processing Toolbox 的 histeq 作单独对照。操作系统为 Windows 10。所有实验图均为 512×512 灰度图，随机种子为 20261008。

图像 1：Alex Guillaume 摄影，Unsplash《An older woman with wrinkles on her face》，原图 3000×3000，暗调皱纹人像，用于直方图实验。来源：https://unsplash.com/photos/an-older-woman-with-wrinkles-on-her-face-mSh9yhcJk1A。

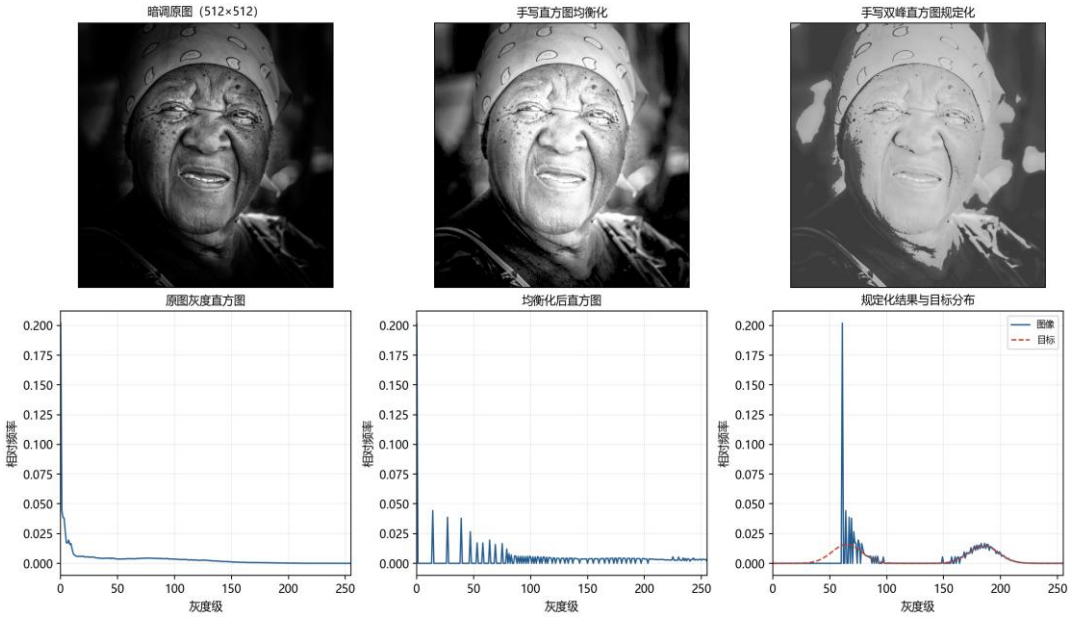
图像 2：Robert Piosik 摄影，Unsplash 灰度女性面部人像，原图 3000×2000，含清晰雀斑，用于边缘与滤波实验。先中心裁剪为 2000×2000，再缩放为 512×512，不拉伸。来源：https://unsplash.com/es/fotos/foto-en-escala-de-grises-de-la-cara-de-la-mujer-bJR4cvkfxf0。许可：https://unsplash.com/license。

任务 1：直方图均衡化与规定化

灰度直方图 H(k)=n_k，相对频率 p(k)=n_k/(MN)，累积分布 CDF(k)=Σ_{j≤k}p(j)。离散均衡化按 s_k=round(255 • (CDF(k)-CDF_min)/(1-CDF_min)) 映射；规定化先求源、目标 CDF，再将源 CDF 映射到第一个不小于该概率的目标灰度级。双峰目标为归一化后的 0.52 • N(65,13²)+0.48 • N(185,13²)。

图像	均值	标准差	P1 - P99	熵(bit)
暗调原图	42.14	48.98	0 - 180	6.252
均衡化	103.25	82.54	0 - 252	6.125
规定化	125.02	59.10	61 - 212	5.486

图 1 暗调原图、手写均衡化、双峰规定化及灰度直方图



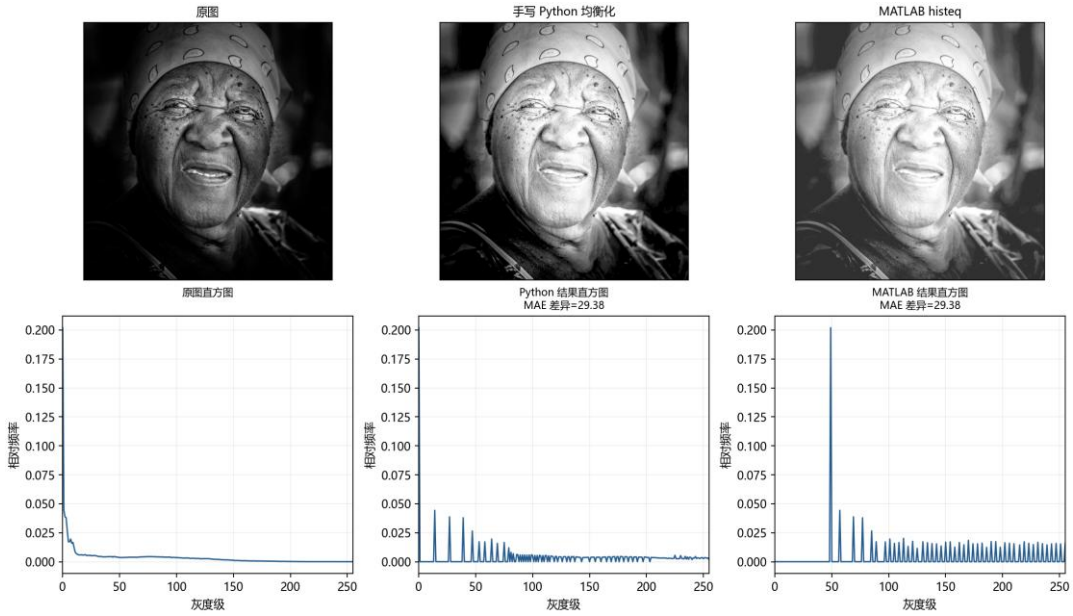
任务 1 结果分析与 MATLAB 对照

暗调原图灰度均值为 42.14，标准差为 48.98，P99 为 180。均衡化后均值为 103.25、标准差为 82.54、P99 为 252；面部暗部层次被展开，但脸部高光明显变亮，背景与皮肤纹理有过增强现象。

规定化后均值为 125.02、标准差为 59.10。双峰目标把大量暗像素映射到约 61 的灰度附近，暗背景被抬亮；面部过渡较均衡，但灰度层级更集中，局部对比度下降。目标分布决定了面部亮度和肤色层次，不能仅凭峰值更多判断视觉质量。

MATLAB R2023b 可用 Image Processing Toolbox 的 histeq；该版本不存在名为 imhisteq 的函数。对同一 512×512 输入运行 histeq 后，与本项目手写实现相比，MAE=29.385，最大绝对差=49。差异来自离散 CDF 量化及映射规则不同;本对照没有替代 Python 实验实现。

图 2 手写 Python 结果与 MATLAB histeq 对照（使用同一输入与灰度显示范围）



任务 3：传统边缘锐化

Roberts 使用 2×2 交叉差分核 Gx=[[1,0],[0,-1]]、Gy=[[0,1],[-1,0]]；Sobel 的水平核为[[-1,0,1],[-2,0,2],[-1,0,1]]，垂直核为其转置方向；Prewitt 将中间行/列权重改为 1。边缘幅值 $E=\sqrt{(Gx^2+Gy^2)}$。代码以 E−mean3×3(E) 作为局部细节掩膜，再计算 clip(I+0.2·D,0,255)，同时输出共同比例的边缘响应。

Laplacian 使用中心为正的四邻域核 L=[[0,-1,0],[-1,4,-1],[0,-1,0]]。亮点中心的响应为正、相邻暗位置为负；因此锐化方向为 I+L。若采用中心为负的相反符号核，则应使用 I−L。

图 3 Roberts、Sobel、Prewitt、Laplacian 响应（各响应采用同一稳健上限）及 Laplacian 叠加结果

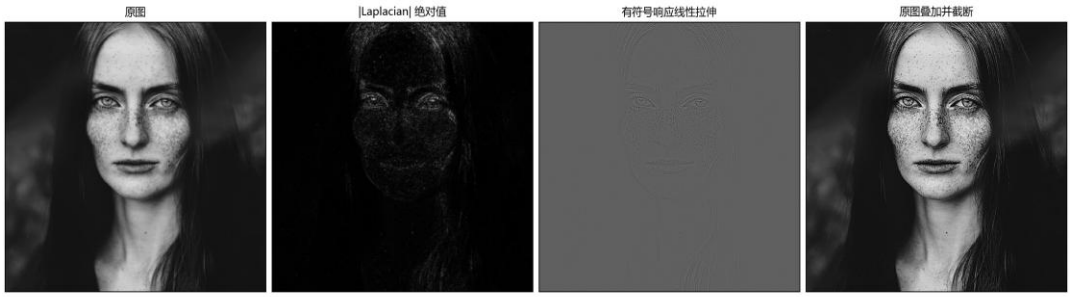


四种算子都能突出眼部、发丝、雀斑和轮廓。Sobel 对中心邻域加权，边缘更连续但响应也更强；Prewitt 权重均匀；Roberts 核小，定位细而对噪声更敏感。边缘锐化只是将局部高频掩膜叠加回原图，幅度过大会放大噪声。

Laplacian 负值处理比较

本实验对同一有符号响应 L 分别采用 |L|、线性拉伸 255·(L−Lmin)/(Lmax−Lmin)，以及 clip(I+L,0,255)。绝对值保留边缘强弱但丢失正负方向；线性拉伸能同时显示两种极性，却把零响应映射到中间灰度，结果是边缘图而非保持原亮度的增强图；原图叠加保留低频外观，并用正负响应提升亮暗边界对比。

图 4 Laplacian 绝对值、线性拉伸和原图叠加截断对比

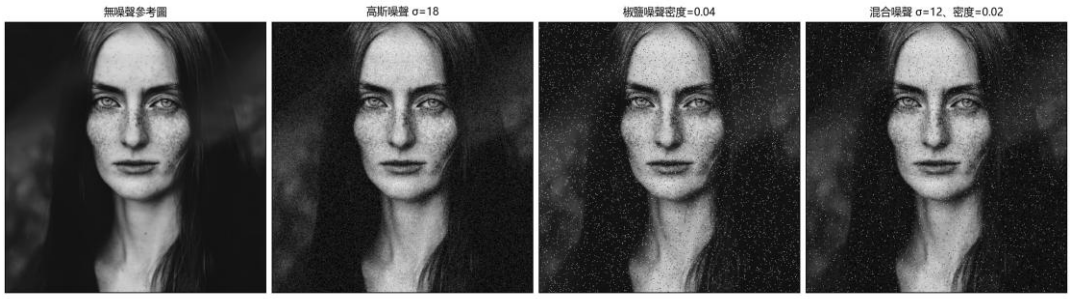


本例中原图叠加最符合增强细节的目标，因为它保留原图结构并叠加带符号的高频变化。截断到[0,255]会在极亮或极暗处损失少量响应；绝对值法和线性拉伸会改变亮度语义，不适合作为原图增强结果直接替代。

任务 4：噪声模型与传统平滑

以雀斑人像为无噪声参考 f。高斯噪声为 g=f+n，n~N(0,σ²)，本次 σ=18；椒盐噪声按总密度 0.04 随机置 0 或 255；混合噪声先加 σ=12 高斯噪声，再以总密度 0.02 加入椒盐点。各随机过程使用固定种子 20261008（子过程使用确定性偏移），便于复现。

图 5 干净参考图与三类固定种子噪声

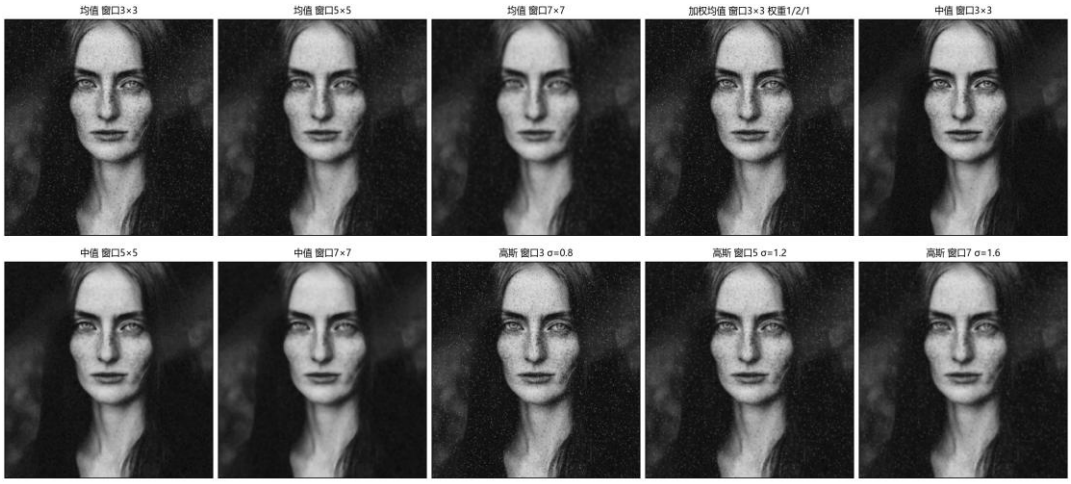


均值滤波 g(x,y)=Σw(i,j)f(x−i,y−j)/Σw，均匀核为所有 w=1；加权核突出中心邻域。中值滤波取窗口样本中位数，适合椒盐脉冲。高斯滤波权重 Gσ(i,j)∝exp(−(i²+j²)/(2σ²))，本实现按两个一维核分离计算。边界使用反射填充，数组先转为 float，最终显示时才裁剪回[0,255]。

传统滤波窗口参数实验

方法	参数	时间(s)	PSNR(dB)	SSIM
均值	窗口 3×3	0.0055	28.778	0.6450
均值	窗口 5×5	0.0052	30.497	0.7671
均值	窗口 7×7	0.0053	30.446	0.8177
加权均值	窗口 3×3 权重 1/2/1	0.0072	28.147	0.6207
中值	窗口 3×3	0.0438	32.236	0.7661
中值	窗口 5×5	0.0905	32.494	0.8502
中值	窗口 7×7	0.1582	31.836	0.8661
高斯	窗口 3 σ=0.8	0.0066	27.937	0.6120
高斯	窗口 5 σ=1.2	0.0087	30.323	0.7349
高斯	窗口 7 σ=1.6	0.0105	31.017	0.8004

图 6 3×3、5×5、7×7 与加权模板的视觉敏感性



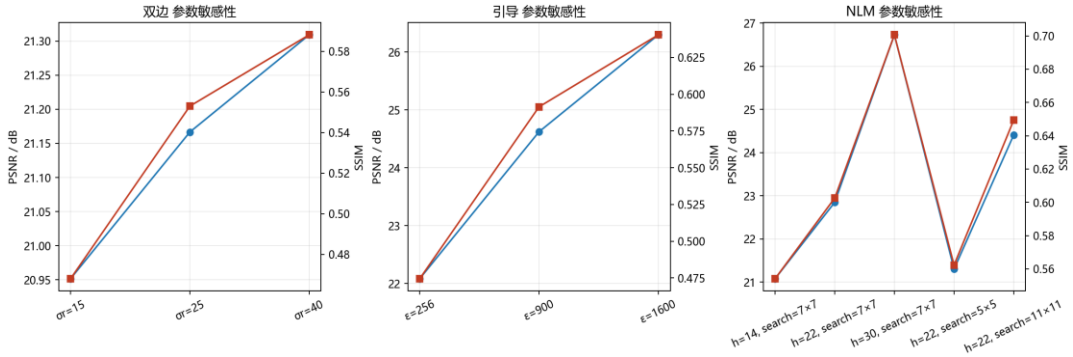
评价对象均为混合噪声结果，PSNR/SSIM 参考加噪前的原始雀斑图。中值 5×5 获得本组最高 PSNR 32.494 dB、SSIM 0.8502，耗时 0.0855 s；7×7 的 SSIM 升至 0.8661 但 PSNR 降至 31.836 dB 且耗时增加，说明大窗口抹平更多高频细节。均值和高斯随窗口变大抑噪增强，同时脸部边缘变软；3×3 加权均值保留局部纹理较多，但本次混合噪声下脉冲残留明显。

现代边缘保持滤波与参数敏感性

双边滤波 $w(p,q)=\exp(-||p-q||^2/(2\sigma s^2))\cdot\exp(-(I(p)-I(q))^2/(2\sigma r^2))$，按空间距离和灰度差共同加权。引导滤波在局部窗口拟合 $q=aI+b$，$a=\mathrm{cov}(I,p)/(\mathrm{var}(I)+\varepsilon)$，$b=\mathrm{mean}(p)-a\cdot\mathrm{mean}(I)$，再平均系数。NLM 以补丁距离 $d^2(p,q)$ 计算 $w=\exp(-\max(d^2-2\sigma n^2,0)/h^2)$，输出为加权平均；本实现逐位移矩阵计算补丁均方差并用积分图盒式均值加速，没有调用现成 NLM。

方法	参数	时间(s)	PSNR(dB)	SSIM
双边	σr=15	0.235	20.952	0.4680
双边	σr=25	0.237	21.167	0.5530
双边	σr=40	0.233	21.309	0.5881
引导	ε =256	0.035	22.084	0.4745
引导	ε =900	0.035	24.623	0.5914
引导	ε =1600	0.037	26.298	0.6404
NLM	h=14, search=7×7	0.500	21.079	0.5541
NLM	h=22, search=7×7	0.505	22.849	0.6026
NLM	h=30, search=7×7	0.505	26.730	0.7006
NLM	h=22, search=5×5	0.261	21.302	0.5623
NLM	h=22, search=11×11	1.425	24.410	0.6494

图 7 双边 σr、引导 ε、NLM h 与搜索窗敏感性



本次 σr=15→40、ε =256→1600、h=14→30 时 PSNR/SSIM 均上升，表示对当前混合噪声更强的平滑提高了与受控干净参考的相似度。NLM 搜索窗从 5×5 扩大到 11×11，h=22 时耗时由实测 0.261 s 增至 1.425 s，指标也变化；成本随搜索位移数增加。所用 NLM 通过每个搜索位移的积分图距离计算，将逐像素 Python 循环改为矩阵化运算，但仍比简单盒式或可分离滤波耗时。

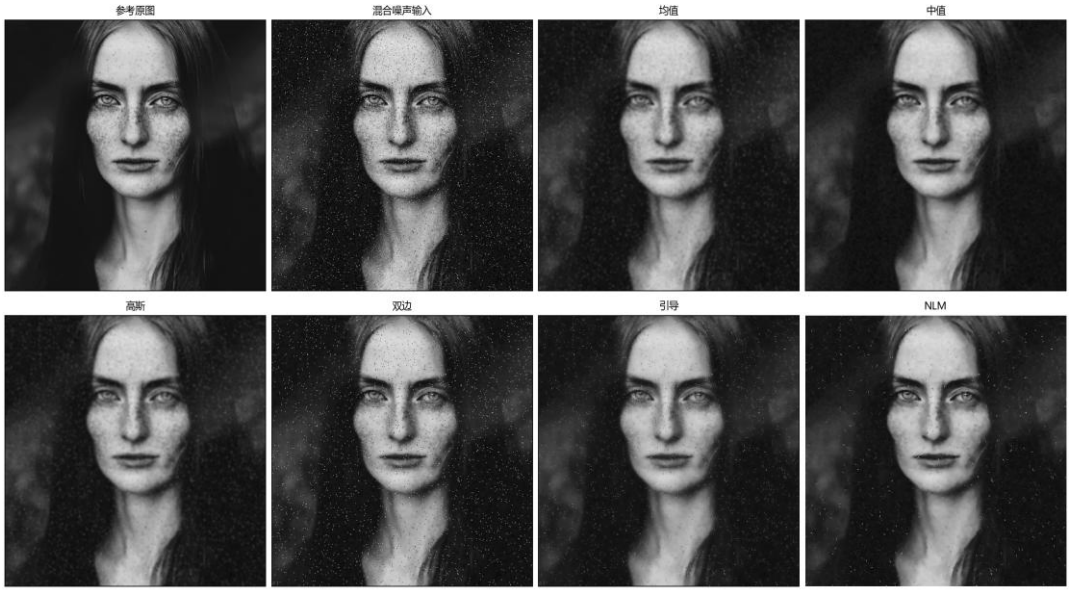
六种滤波算法综合对比

下表所有指标都来自同一张 512×512 混合噪声图;PSNR/SSIM 以加噪前人像作为干净参考。耗时为本机 perf_counter 单次实测，不是理论值。复杂度按像素数 N、核宽 k、NLM 搜索边长 s 表示；引导与均值使用积分图盒均值。

指标定义：$MSE=(1/MN)\Sigma(I-K)^2$，$PSNR=10\log_{10}(255^2/MSE)$；$SSIM=((2\mu_I\mu_K+C_1)(2\sigma_{IK}+C_2))/((\mu_I^2+\mu_K^2+C_1)(\sigma_I^2+\sigma_K^2+C_2))$，均值、方差和协方差在局部窗口计算，$C_1$、$C_2$ 用于稳定分母。

方法	参数	时间(s)	PSNR(dB)	SSIM	理论复杂度
均值	5×5	0.009	30.497	0.7671	O(N) integral box filter
中值	5×5	0.085	32.494	0.8502	O(N k^2) neighborhood selection
高斯	7×7, σ=1.6	0.010	31.017	0.8004	O(N k) separable
双边	r=3, σ_s=2, σ_r=40	0.257	21.309	0.5881	O(N k^2)
引导	r=3, ε=1600	0.035	26.298	0.6404	O(N) integral box filters
NLM	patch=5×5, search=7×7, h=30	0.493	26.730	0.7006	O(N s^2) search displacements

图 8 六种滤波结果（统一灰度范围）



本组中值 5×5 的 PSNR/SSIM 最高；均值 5×5 速度最快，但细节较软。高斯 7×7 在抑噪和结构保留间折中。双边滤波本组指标偏低，原因是强脉冲噪声不符合其局部灰度相似假设，调大 σ_r 不能完全消除脉冲;引导滤波和 NLM 的指标随 ε/h 增大改善,但纹理也更平滑。指标排序依赖噪声类型、参数与参考图，不代表所有图像的通用优劣。

有参考去噪与人脸平滑的区别

加噪实验保留了真实拍摄的人像作为人工加噪前的像素参考，因此该实验的 PSNR/SSIM 衡量恢复到输入照片的程度。美容平滑实验直接处理带雀斑的人像；原图包含真实雀斑和皮肤纹理，并非无瑕疵真值，所以该实验不报告 PSNR/SSIM。

图 9 对真实雀斑人像进行六种平滑的无参考视觉比较

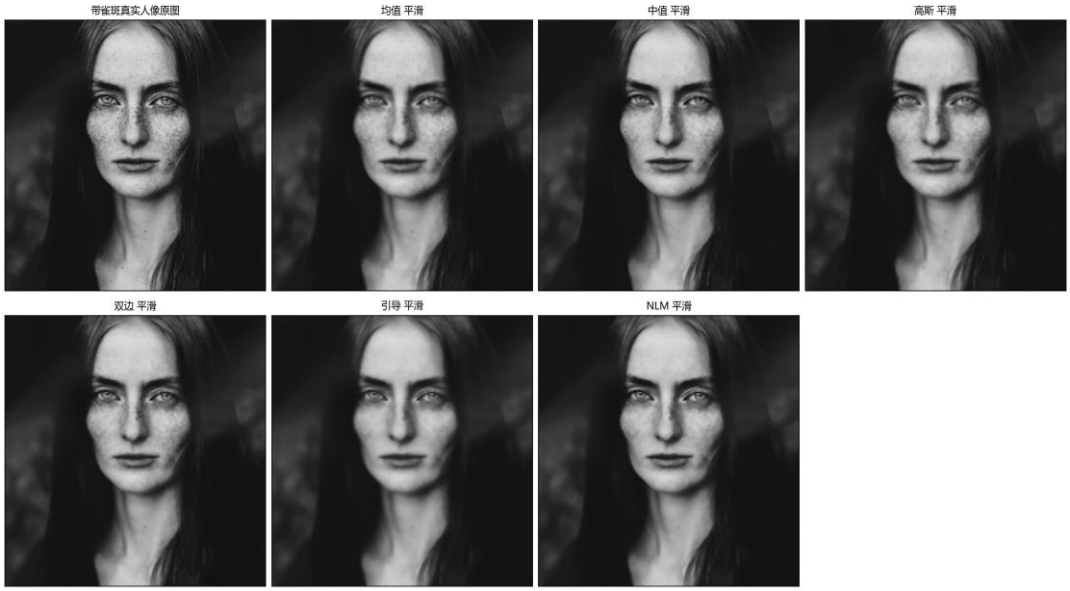


图 10 混合噪声去噪的 128×128 局部对比



均值与高斯会一起削弱肤色细纹和雀斑；中值更能抑制孤立点，但窗口变大也会损失细纹；双边、引导与 NLM 在边缘附近保留较多结构，参数过大时同样会让斑点和纹理变淡。视觉上更光滑不等于恢复了真实无瑕疵皮肤。

实验总结：掌握并实现了直方图映射、传统边缘锐化和多类平滑。调试解决了 uint8 溢出、Laplacian 符号、反射边界和 NLM 计算量问题；结果说明滤波效果取决于噪声类型、参数与参考图，任务 1、3、4 均完成。

附录 A 核心运行源码：dip_lab/histogram.py

```
"""Hand-written grayscale histogram operations for uint8 images."""

from typing import Union

import numpy as np


ArrayLike = Union[np.ndarray, list]


def _gray_u8(image: ArrayLike) -> np.ndarray:
    array = np.asarray(image)
    if array.ndim != 2:
        raise ValueError("Expected a two-dimensional grayscale image")
    if not np.isfinite(array).all():
        raise ValueError("Image values must be finite")
    return np.clip(np.rint(array), 0, 255).astype(np.uint8)


def gray_histogram(image: ArrayLike) -> np.ndarray:
    """Count the occurrences of gray levels 0..255 without image libraries."""
    pixels = _gray_u8(image).ravel()
    return np.bincount(pixels, minlength=256).astype(np.int64, copy=False)


def equalize_histogram(image: ArrayLike) -> np.ndarray:
    """Apply discrete CDF equalization, mapping the first occupied CDF to zero."""
    source = _gray_u8(image)
    counts = gray_histogram(source)
    occupied = np.flatnonzero(counts)
    if occupied.size <= 1:
        return source.copy()

    cdf = counts.cumsum(dtype=np.float64) / float(source.size)
    cdf_min = cdf[occupied[0]]
    denominator = 1.0 - cdf_min
    if denominator <= np.finfo(np.float64).eps:
        return source.copy()
    lut = np.clip(np.rint((cdf - cdf_min) * (255.0 / denominator)), 0, 255)
    return lut[source].astype(np.uint8)


def _target_probability(target: ArrayLike) -> np.ndarray:
    values = np.asarray(target)
```

```
    if values.ndim == 2:
        counts = gray_histogram(values)
        return counts.astype(np.float64) / max(float(counts.sum()), 1.0)
    if values.ndim != 1 or values.size != 256:
        raise ValueError("Target must be a grayscale image or a 256-entry histogram")
    probability = values.astype(np.float64)
    if not np.isfinite(probability).all() or np.any(probability < 0):
        raise ValueError("Target histogram must contain finite, non-negative values")
    total = float(probability.sum())
    if total <= 0:
        raise ValueError("Target histogram must have positive mass")
    return probability / total


def match_histogram(image: ArrayLike, target: ArrayLike) -> np.ndarray:
    """Map source CDF values to the closest target CDF gray level."""
    source = _gray_u8(image)
    source_counts = gray_histogram(source)
    if np.count_nonzero(source_counts) <= 1:
        return source.copy()

    source_cdf = source_counts.cumsum(dtype=np.float64) / float(source.size)
    target_cdf = np.cumsum(_target_probability(target), dtype=np.float64)
    target_cdf[-1] = 1.0
    lut = np.searchsorted(target_cdf, source_cdf, side="left")
    lut = np.clip(lut, 0, 255).astype(np.uint8)
    return lut[source]


def make_bimodal_target(
    peak1: float = 65.0,
    peak2: float = 185.0,
    sigma: float = 12.0,
    weight1: float = 0.5,
) -> np.ndarray:
    """Build a normalized two-Gaussian target distribution over gray levels."""
    if sigma <= 0:
        raise ValueError("sigma must be positive")
    if not 0.0 <= weight1 <= 1.0:
        raise ValueError("weight1 must be in [0, 1]")
    levels = np.arange(256, dtype=np.float64)
    component1 = np.exp(-0.5 * ((levels - peak1) / sigma) ** 2)
    component2 = np.exp(-0.5 * ((levels - peak2) / sigma) ** 2)
    target = weight1 * component1 + (1.0 - weight1) * component2
    return target / target.sum()
```

附录 A 核心运行源码：dip_lab/spatial.py

```
"""Manual spatial-domain filters and classical edge operators."""

from typing import Optional

import numpy as np



def _float_gray(image: np.ndarray) -> np.ndarray:
    array = np.asarray(image, dtype=np.float64)
    if array.ndim != 2:
        raise ValueError("Expected a two-dimensional grayscale image")
    if not np.isfinite(array).all():
        raise ValueError("Image values must be finite")
    return array



def _check_odd_size(size: int) -> int:
    size = int(size)
    if size < 1 or size % 2 == 0:
        raise ValueError("Window size must be a positive odd integer")
    return size



def _pad_mode(image: np.ndarray) -> str:
    return "symmetric" if min(image.shape) == 1 else "reflect"



def correlate2d(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Correlate a grayscale image with an odd-sized kernel using NumPy only."""
    source = _float_gray(image)
    weights = np.asarray(kernel, dtype=np.float64)
    if weights.ndim != 2 or any(size % 2 == 0 for size in weights.shape):
        raise ValueError("Kernel dimensions must be odd")
    row_radius, col_radius = weights.shape[0] // 2, weights.shape[1] // 2
    padded = np.pad(
        source,
        ((row_radius, row_radius), (col_radius, col_radius)),
        mode=_pad_mode(source),
    )
    result = np.zeros_like(source)
    height, width = source.shape
    for row in range(weights.shape[0]):
        for col in range(weights.shape[1]):
            result += weights[row, col] * padded[row : row + height, col : col + width]
```

```
    return result


def box_filter(image: np.ndarray, size: int) -> np.ndarray:
    """Return local box means using an integral image and reflected borders."""
    source = _float_gray(image)
    size = _check_odd_size(size)
    radius = size // 2
    padded = np.pad(source, ((radius, radius), (radius, radius)), mode=_pad_mode(source))
    integral = np.pad(padded.cumsum(axis=0).cumsum(axis=1), ((1, 0), (1, 0)))
    height, width = source.shape
    bottom_right = integral[size : size + height, size : size + width]
    top_right = integral[:height, size : size + width]
    bottom_left = integral[size : size + height, :width]
    top_left = integral[:height, :width]
    return (bottom_right - top_right - bottom_left + top_left) / float(size * size)


def mean_filter(
    image: np.ndarray,
    size: int = 3,
    weights: Optional[np.ndarray] = None,
) -> np.ndarray:
    """Apply a uniform or caller-supplied normalized spatial mean kernel."""
    source = _float_gray(image)
    if weights is None:
        return box_filter(source, size)
    kernel = np.asarray(weights, dtype=np.float64)
    if kernel.ndim != 2 or any(dimension % 2 == 0 for dimension in kernel.shape):
        raise ValueError("weights must be a two-dimensional odd-sized matrix")
    total = float(kernel.sum())
    if not np.isfinite(kernel).all() or abs(total) <= np.finfo(np.float64).eps:
        raise ValueError("weights must have a finite, non-zero sum")
    return correlate2d(source, kernel / total)


def median_filter(image: np.ndarray, size: int = 3) -> np.ndarray:
    """Apply a sliding-window median with reflected borders."""
    source = _float_gray(image)
    size = _check_odd_size(size)
    radius = size // 2
    padded = np.pad(source, ((radius, radius), (radius, radius)), mode=_pad_mode(source))
    windows = np.lib.stride_tricks.sliding_window_view(padded, (size, size))
    return np.median(windows, axis=(-2, -1))
```

```
def _filter_axis(image: np.ndarray, kernel: np.ndarray, axis: int) -> np.ndarray:
    radius = kernel.size // 2
    pad_width = ((radius, radius), (0, 0)) if axis == 0 else ((0, 0), (radius, radius))
    padded = np.pad(image, pad_width, mode=_pad_mode(image))
    result = np.zeros_like(image)
    height, width = image.shape
    for index, weight in enumerate(kernel):
        if axis == 0:
            result += weight * padded[index : index + height, :]
        else:
            result += weight * padded[:, index : index + width]
    return result


def gaussian_filter(image: np.ndarray, size: int = 5, sigma: float = 1.0) -> np.ndarray:
    """Apply a normalized separable Gaussian kernel without a library filter."""
    source = _float_gray(image)
    size = _check_odd_size(size)
    if sigma <= 0:
        raise ValueError("sigma must be positive")
    coordinates = np.arange(-(size // 2), size // 2 + 1, dtype=np.float64)
    kernel = np.exp(-(coordinates**2) / (2.0 * sigma**2))
    kernel /= kernel.sum()
    return _filter_axis(_filter_axis(source, kernel, axis=1), kernel, axis=0)


def roberts_components(image: np.ndarray) -> tuple:
    """Return two signed Roberts cross responses using reflected bottom/right edges."""
    source = _float_gray(image)
    height, width = source.shape
    padded = np.pad(source, ((0, 1), (0, 1)), mode=_pad_mode(source))
    gx = padded[:height, :width] - padded[1 : height + 1, 1 : width + 1]
    gy = padded[:height, 1 : width + 1] - padded[1 : height + 1, :width]
    return gx, gy


def _gradient_components(image: np.ndarray, kind: str) -> tuple:
    if kind == "sobel":
        horizontal = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float64)
        vertical = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=np.float64)
    elif kind == "prewitt":
        horizontal = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float64)
        vertical = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float64)
    else:
```

```
        raise ValueError("kind must be 'sobel' or 'prewitt'")
    return correlate2d(image, horizontal), correlate2d(image, vertical)


def _magnitude(components: tuple) -> np.ndarray:
    gx, gy = components
    return np.hypot(gx, gy)


def roberts_response(image: np.ndarray) -> np.ndarray:
    return _magnitude(roberts_components(image))


def sobel_components(image: np.ndarray) -> tuple:
    return _gradient_components(image, "sobel")


def sobel_response(image: np.ndarray) -> np.ndarray:
    return _magnitude(sobel_components(image))


def prewitt_components(image: np.ndarray) -> tuple:
    return _gradient_components(image, "prewitt")


def prewitt_response(image: np.ndarray) -> np.ndarray:
    return _magnitude(prewitt_components(image))


def laplacian_response(image: np.ndarray) -> np.ndarray:
    """Signed four-neighbor Laplacian with a positive center coefficient."""
    kernel = np.array([[0, -1, 0], [-1, 4, -1], [0, -1, 0]], dtype=np.float64)
    return correlate2d(image, kernel)


def stretch_to_u8(image: np.ndarray) -> np.ndarray:
    """Linearly stretch finite values to [0, 255], returning zeros if constant."""
    source = _float_gray(image)
    low, high = float(source.min()), float(source.max())
    if high - low <= np.finfo(np.float64).eps:
        return np.zeros(source.shape, dtype=np.uint8)
    return np.clip(np.rint((source - low) * (255.0 / (high - low))), 0, 255).astype(np.uint8)


def laplacian_variants(image: np.ndarray) -> dict:
```

```
    """Return absolute, stretched, additive-clipped, and raw signed Laplacian results."""
    source = _float_gray(image)
    response = laplacian_response(source)
    return {
        "response": response,
        "absolute": np.clip(np.rint(np.abs(response)), 0, 255).astype(np.uint8),
        "stretched": stretch_to_u8(response),
        "overlay": np.clip(np.rint(source + response), 0, 255).astype(np.uint8),
    }


def edge_sharpen(image: np.ndarray, kind: str, amount: float = 0.15) -> np.ndarray:
    """Create a conservative edge-mask overlay; signed Laplacian is preferred for detail."""
    source = _float_gray(image)
    if kind == "roberts":
        response = roberts_response(source)
    elif kind == "sobel":
        response = sobel_response(source)
    elif kind == "prewitt":
        response = prewitt_response(source)
    else:
        raise ValueError("kind must be roberts, sobel, or prewitt")
    # Centering the positive magnitude limits a global brightness lift while retaining edges.
    detail = response - box_filter(response, 3)
    return np.clip(source + amount * detail, 0.0, 255.0)
```

附录 A 核心运行源码：dip_lab/noise.py

```
"""Reproducible Gaussian, impulse, and mixed noise generation."""

import numpy as np


def _gray_u8(image: np.ndarray) -> np.ndarray:
    array = np.asarray(image)
    if array.ndim != 2 or not np.isfinite(array).all():
        raise ValueError("Expected a finite two-dimensional grayscale image")
    return np.clip(np.rint(array), 0, 255).astype(np.uint8)


def add_gaussian_noise(image: np.ndarray, sigma: float = 18.0, seed: int = 20261008) -> np.ndarray:
    """Add zero-mean Gaussian noise, clip to display range, and return uint8."""
    source = _gray_u8(image)
    if sigma < 0:
        raise ValueError("sigma must be non-negative")
    if sigma == 0:
        return source.copy()
    rng = np.random.default_rng(seed)
    noisy = source.astype(np.float64) + rng.normal(0.0, sigma, size=source.shape)
    return np.clip(np.rint(noisy), 0, 255).astype(np.uint8)


def add_salt_pepper_noise(
    image: np.ndarray,
    density: float = 0.04,
    seed: int = 20261008,
) -> np.ndarray:
    """Replace a reproducible fraction of pixels with 0 or 255."""
    source = _gray_u8(image)
    if not 0.0 <= density <= 1.0:
        raise ValueError("density must be in [0, 1]")
    if density == 0:
        return source.copy()
    rng = np.random.default_rng(seed)
    draw = rng.random(source.shape)
    result = source.copy()
    result[draw < density / 2.0] = 0
    result[(draw >= density / 2.0) & (draw < density)] = 255
    return result


def add_mixed_noise(
```

```
    image: np.ndarray,
    gaussian_sigma: float = 12.0,
    salt_pepper_density: float = 0.02,
    seed: int = 20261008,
) -> np.ndarray:
    """Add Gaussian and salt-pepper noise from independent fixed RNG streams."""
    source = _gray_u8(image)
    if gaussian_sigma < 0 or not 0.0 <= salt_pepper_density <= 1.0:
        raise ValueError("Noise parameters are outside their valid ranges")
    if gaussian_sigma == 0 and salt_pepper_density == 0:
        return source.copy()
    rng = np.random.default_rng(seed)
    noisy = source.astype(np.float64)
    if gaussian_sigma > 0:
        noisy += rng.normal(0.0, gaussian_sigma, size=source.shape)
    result = np.clip(np.rint(noisy), 0, 255).astype(np.uint8)
    if salt_pepper_density > 0:
        draw = rng.random(source.shape)
        result[draw < salt_pepper_density / 2.0] = 0
        result[(draw >= salt_pepper_density / 2.0) & (draw < salt_pepper_density)] = 255
    return result
```

附录 A 核心运行源码：dip_lab/advanced.py

```
"""Manual edge-preserving filters implemented with NumPy array operations."""

import math

import numpy as np

from .spatial import box_filter


def _gray(image: np.ndarray) -> np.ndarray:
    array = np.asarray(image, dtype=np.float64)
    if array.ndim != 2 or not np.isfinite(array).all():
        raise ValueError("Expected a finite two-dimensional grayscale image")
    return array


def _odd_positive(value: int, label: str) -> int:
    value = int(value)
    if value < 1:
        raise ValueError(label + " must be positive")
    return value


def _shift_reflect(image: np.ndarray, dy: int, dx: int) -> np.ndarray:
    """Translate an image while reflecting values beyond its boundary."""
    source = _gray(image)
    height, width = source.shape
    mode = "symmetric" if min(source.shape) == 1 else "reflect"
    pad_y, pad_x = abs(dy), abs(dx)
    padded = np.pad(source, ((pad_y, pad_y), (pad_x, pad_x)), mode=mode)
    start_y = pad_y + dy
    start_x = pad_x + dx
    return padded[start_y : start_y + height, start_x : start_x + width]


def bilateral_filter(
    image: np.ndarray,
    radius: int = 3,
    sigma_space: float = 2.0,
    sigma_range: float = 25.0,
) -> np.ndarray:
    """Apply bilateral weighting over a square neighborhood; no library filter calls."""
    source = _gray(image)
    radius = _odd_positive(radius, "radius")
```

```
    if sigma_space <= 0 or sigma_range <= 0:
        raise ValueError("sigma_space and sigma_range must be positive")
    numerator = np.zeros_like(source)
    denominator = np.zeros_like(source)
    spatial_denominator = 2.0 * sigma_space**2
    range_denominator = 2.0 * sigma_range**2
    for dy in range(-radius, radius + 1):
        for dx in range(-radius, radius + 1):
            neighbor = _shift_reflect(source, dy, dx)
            spatial_weight = math.exp(-(dy * dy + dx * dx) / spatial_denominator)
            range_weight = np.exp(-((neighbor - source) ** 2) / range_denominator)
            weight = spatial_weight * range_weight
            numerator += weight * neighbor
            denominator += weight
    return numerator / np.maximum(denominator, np.finfo(np.float64).tiny)



def guided_filter(
    guide: np.ndarray,
    source: np.ndarray,
    radius: int = 4,
    epsilon: float = 650.25,
) -> np.ndarray:
    """Grayscale guided filter using local linear coefficients and integral box means."""
    guide_image = _gray(guide)
    source_image = _gray(source)
    if guide_image.shape != source_image.shape:
        raise ValueError("guide and source must have equal shapes")
    radius = _odd_positive(radius, "radius")
    if epsilon <= 0:
        raise ValueError("epsilon must be positive")
    window = 2 * radius + 1
    mean_i = box_filter(guide_image, window)
    mean_p = box_filter(source_image, window)
    mean_ip = box_filter(guide_image * source_image, window)
    mean_ii = box_filter(guide_image * guide_image, window)
    covariance = mean_ip - mean_i * mean_p
    variance = np.maximum(mean_ii - mean_i * mean_i, 0.0)
    a = covariance / (variance + epsilon)
    b = mean_p - a * mean_i
    mean_a = box_filter(a, window)
    mean_b = box_filter(b, window)
    return mean_a * guide_image + mean_b
```

```
def non_local_means(
    image: np.ndarray,
    patch_radius: int = 2,
    search_radius: int = 5,
    h: float = 18.0,
    noise_sigma: float = 0.0,
) -> np.ndarray:
    """Approximate NLM with vectorized shifted images and integral patch distances.

    The search still examines every displacement in the specified square window.
    Patch SSD is computed as a box mean, avoiding per-pixel Python loops.
    """
    source = _gray(image)
    patch_radius = _odd_positive(patch_radius, "patch_radius")
    search_radius = _odd_positive(search_radius, "search_radius")
    if h <= 0 or noise_sigma < 0:
        raise ValueError("h must be positive and noise_sigma non-negative")
    patch_size = 2 * patch_radius + 1
    numerator = np.zeros_like(source)
    denominator = np.zeros_like(source)
    noise_bias = 2.0 * noise_sigma**2
    for dy in range(-search_radius, search_radius + 1):
        for dx in range(-search_radius, search_radius + 1):
            neighbor = _shift_reflect(source, dy, dx)
            if dy == 0 and dx == 0:
                weight = np.ones_like(source)
            else:
                distance = box_filter((source - neighbor) ** 2, patch_size)
                corrected = np.maximum(distance - noise_bias, 0.0)
                weight = np.exp(-corrected / (h * h))
            numerator += weight * neighbor
            denominator += weight
    return numerator / np.maximum(denominator, np.finfo(np.float64).tiny)
```

附录 A 核心运行源码：dip_lab/metrics.py

```
"""Quality metrics for denoising with an explicitly available clean reference."""

import numpy as np
from skimage.metrics import peak_signal_noise_ratio, structural_similarity


def psnr(reference: np.ndarray, estimate: np.ndarray) -> float:
    """PSNR on the shared 0..255 grayscale scale."""
    ref = np.asarray(reference, dtype=np.float64)
    est = np.asarray(estimate, dtype=np.float64)
    if ref.shape != est.shape or ref.ndim != 2:
        raise ValueError("reference and estimate must be same-shaped grayscale images")
    if np.array_equal(ref, est):
        return float("inf")
    return float(peak_signal_noise_ratio(ref, est, data_range=255.0))


def ssim(reference: np.ndarray, estimate: np.ndarray) -> float:
    """SSIM using scikit-image's validated metric implementation."""
    ref = np.asarray(reference, dtype=np.float64)
    est = np.asarray(estimate, dtype=np.float64)
    if ref.shape != est.shape or ref.ndim != 2:
        raise ValueError("reference and estimate must be same-shaped grayscale images")
    if min(ref.shape) < 7:
        raise ValueError("SSIM requires both dimensions to be at least 7 pixels")
    return float(structural_similarity(ref, est, data_range=255.0, channel_axis=None))


def compare(reference: np.ndarray, estimate: np.ndarray) -> dict:
    return {"psnr_db": psnr(reference, estimate), "ssim": ssim(reference, estimate)}
```